

ChangeOrder.Api

Change Request Management System

PROGRAMMER PROJECT GUIDE — GUIA TECNICA DEL PROYECTO

Version	1.0.0
Fecha	2026-03-10
Target	.NET 10 LTS C# 14 Minimal APIs
Clasificacion	Uso Interno — Confidencial

1. Resumen del Proyecto

Proposito del Sistema

Sistema de Control de Ordenes de Cambio (Change Request Management). Cuando un cliente solicita un cambio a una aplicacion en produccion, se genera un numero de orden con formato: yyyyMMdd-## (Ejemplo: 20260224-01).

El sistema es un CRUD completo expuesto como WebAPI con Minimal APIs. El objetivo es gestionar el ciclo de vida completo de las solicitudes de cambio desde su creacion hasta el despliegue en produccion, incluyendo un flujo de aprobacion en cadena de 4 niveles.

Formato del Numero de Orden

```
Formato:  yyyyMMdd-##  
Ejemplo:  20260224-01  
          |_____|_|  
          Fecha  Seq
```

Fase 2 (Futura)

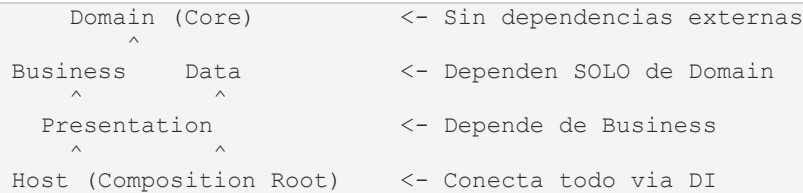
Cliente WPF con MVVM (CommunityToolkit.Mvvm, MaterialDesignInXaml) que consume esta API via HttpClient para CRUD completo y genera documentos/reportes por numero de orden. Las reglas del cliente WPF se definiran en un documento separado.

2. Stack Tecnologico

Tecnologia	Version / Detalle	Proposito
.NET	10 (LTS — soporte hasta Nov 2028)	Framework principal
C#	14	Lenguaje de programacion
ASP.NET Core	10 — Minimal APIs	API REST
Entity Framework Core	10 — Code-First	ORM / Base de datos
SQL Server	MSSQL	Motor de base de datos
Serilog	Ultima version estable	Logging estructurado
Swagger / OpenAPI	3.1 (default .NET 10)	Documentacion de API
Docker	Multi-stage build	Contenedorizacion
GitHub Actions	CI/CD	Integracion y despliegue continuo

3. Arquitectura — Onion Architecture

El proyecto usa Onion Architecture (también conocida como Clean Architecture). Las capas internas no dependen de las externas. El dominio es el núcleo sin dependencias externas.



Proyectos de la Solucion

Proyecto	Responsabilidad	Referencias
ChangeOrder.Domain	Entidades, Value Objects, interfaces, enums	Ninguna
ChangeOrder.Business	Servicios, Handlers CQRS, validaciones	Domain
ChangeOrder.Data	DbContext, Repositories, Migrations	Domain
ChangeOrder.Presentation	Endpoints Minimal API, DTOs, Mappers	Business
ChangeOrder.Host	Program.cs, DI, configuracion	Presentation, Data

Reglas de Dependencia — CRITICO

⚠ CRITICO: Estas reglas son INVIOLABLES. Violarlas rompe la arquitectura y debe ser motivo de rechazo en PR.

- Domain NO referencia ningun otro proyecto.
- Business y Data SOLO referencian Domain.
- Presentation SOLO referencia Business.
- Host es el Composition Root: registra servicios y conecta todas las capas.
- NUNCA referenciar capas superiores desde capas inferiores.

4. Estructura de Carpetas

La siguiente estructura de directorios es obligatoria. No se deben crear archivos fuera de su capa correspondiente.

```
ChangeOrder.sln
|
+-- src/
|   +-- ChangeOrder.Domain/
|   |   +-- Entities/
|   |   |   +-- ChangeOrder.cs
|   |   |   +-- ValueObjects/
|   |   |   |   +-- OrderNumber.cs
|   |   |   |   +-- RequesterInfo.cs
|   |   |   |   +-- ApprovalChain.cs
|   |   |   +-- Enums/
|   |   |   |   +-- ApprovalStatus.cs
|   |   |   |   +-- OrderStatus.cs
|   |   |   +-- Abstractions/
|   |   |   |   +-- IChangeOrderRepository.cs
|   |   |   |   +-- IUnitOfWork.cs
|   |   |   +-- Errors/
|   |   |   |   +-- DomainErrors.cs
|   |   +-- ChangeOrder.Business/
|   |   |   +-- Commands/
|   |   |   |   +-- CreateOrder/
|   |   |   |   |   +-- CreateOrderCommand.cs
|   |   |   |   |   +-- CreateOrderHandler.cs
|   |   |   |   +-- UpdateOrder/
|   |   |   |   +-- DeleteOrder/
|   |   |   +-- Queries/
|   |   |   |   +-- GetOrderById/
|   |   |   |   +-- GetAllOrders/
|   |   |   |   +-- GetOrdersByDate/
|   |   |   +-- Abstractions/
|   |   |   |   +-- ICommandHandler.cs
|   |   |   |   +-- IQueryHandler.cs
|   |   |   +-- Services/
|   |   |   |   +-- OrderNumberGenerator.cs
|   |   +-- ChangeOrder.Data/
|   |   |   +-- Context/
|   |   |   |   +-- ChangeOrderDbContext.cs
|   |   |   +-- Configurations/
|   |   |   |   +-- ChangeOrderConfiguration.cs
|   |   |   +-- Repositories/
|   |   |   |   +-- ChangeOrderRepository.cs
|   |   |   +-- Migrations/
|   |   |   +-- Extensions/
|   |   |   |   +-- ServiceCollectionExtensions.cs
|   |   +-- ChangeOrder.Presentation/
|   |   |   +-- Endpoints/
|   |   |   |   +-- ChangeOrderEndpoints.cs
|   |   |   +-- DTOs/
|   |   |   |   +-- Requests/ (CreateOrderRequest, UpdateOrderRequest)
|   |   |   |   +-- Responses/ (OrderResponse, OrderListResponse)
|   |   |   +-- Mappers/
|   |   |   |   +-- OrderMapper.cs
|   |   |   +-- Extensions/
|   |   |   |   +-- ServiceCollectionExtensions.cs
|   |   +-- ChangeOrder.Host/
|   |   |   +-- Program.cs
```

```
|      +-- appsettings.json
|      +-- appsettings.Development.json
|      +-- Dockerfile
|      +-- Extensions/
|          +-- ServiceCollectionExtensions.cs
+-- tests/
|   +-- ChangeOrder.Domain.Tests/
|   +-- ChangeOrder.Business.Tests/
|   +-- ChangeOrder.Data.Tests/
|   +-- ChangeOrder.Presentation.Tests/
+-- .github/
|   +-- workflows/
|       +-- ci.yml
+-- .gitignore
+-- .editorconfig
+-- Directory.Build.props
+-- README.md
```

5. Reglas de Código C#

10.1 Configuración de Entidad — Mapeado Completo

La configuración COMPLETA de ChangeOrderConfiguration debe mapear TODOS los campos de la entidad, incluyendo Value Objects, enums, soft delete y auditoria:

```
public class ChangeOrderConfiguration
    : IEntityTypeConfiguration<ChangeOrderEntity>
{
    public void Configure(EntityTypeBuilder<ChangeOrderEntity> builder)
    {
        builder.ToTable("ChangeOrders");
        builder.HasKey(x => x.Id);

        // --- Numero de orden (Value Object) ---
        builder.OwnsOne(x => x.Number, n =>
        {
            n.Property(p => p.Value)
                .HasColumnName("OrderNumber")
                .HasMaxLength(13)
                .IsRequired();
            n.HasIndex(p => p.Value).IsUnique();
        });

        // --- Informacion del programa ---
        builder.Property(x => x.ProgramName)
            .HasMaxLength(200).IsRequired();
        builder.Property(x => x.ProductionVersion)
            .HasMaxLength(50).IsRequired();
        builder.Property(x => x.VersionScreenshotPath)
            .HasMaxLength(500);

        // --- Solicitud ---
        builder.Property(x => x.RequestDate).IsRequired();
        builder.Property(x => x.WorkDescription)
            .HasMaxLength(500).IsRequired();
        builder.Property(x => x.RequestDetails)
            .HasColumnType("nvarchar(max)").IsRequired();
        builder.Property(x => x.Justification)
            .HasColumnType("nvarchar(max)").IsRequired();
        builder.Property(x => x.RequiredAction)
            .HasColumnType("nvarchar(max)").IsRequired();

        // --- Solicitante (Value Object) ---
        builder.OwnsOne(x => x.Requester, r =>
        {
            r.Property(p => p.Name).HasMaxLength(150).IsRequired();
            r.Property(p => p.Position).HasMaxLength(100).IsRequired();
            r.Property(p => p.Department).HasMaxLength(100).IsRequired();
            r.Property(p => p.Email).HasMaxLength(200).IsRequired();
        });

        // --- Aprobaciones (Value Object) ---
        builder.OwnsOne(x => x.Approvals, a =>
        {
            a.Property(p => p.RequesterApproval)
                .HasConversion<string>().HasMaxLength(20).IsRequired();
            a.Property(p => p.DepartmentHeadApproval)
```

```
        .HasConversion<string>().HasMaxLength(20).IsRequired();
a.Property(p => p.ItHeadApproval)
    .HasConversion<string>().HasMaxLength(20).IsRequired();
a.Property(p => p.ProgrammingDivisionApproval)
    .HasConversion<string>().HasMaxLength(20).IsRequired();
});

// --- Estado y fechas de seguimiento ---
builder.Property(x => x.Status)
    .HasConversion<string>().HasMaxLength(30).IsRequired();
builder.Property(x => x.DeliveryDate);
builder.Property(x => x.InitialEvaluationDate);
builder.Property(x => x.ProductionDeployDate);
builder.Property(x => x.PostDeployScreenshotPath)
    .HasMaxLength(500);

// --- Auditoria ---
builder.Property(x => x.CreatedAt).IsRequired();
builder.Property(x => x.UpdatedAt);

// --- Soft Delete ---
builder.Property(x => x.IsDeleted).IsRequired().HasDefaultValue(false);
builder.Property(x => x.DeletedAt);
builder.HasQueryFilter(x => !x.IsDeleted);
}
}
```


10.2 AuditInterceptor — Implementacion Completa

⚠ CRITICO: El AuditInterceptor es OBLIGATORIO. Sin el, los campos UpdatedAt, IsDeleted y DeletedAt NO se actualizaran automaticamente.

```
namespace ChangeOrder.Data.Interceptors;

public sealed class AuditInterceptor : SaveChangesInterceptor
{
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData,
        InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        DbContext? context = eventData.Context;
        if (context is null)
            return base.SavingChangesAsync(eventData, result, cancellationToken);

        DateTime now = DateTime.UtcNow;

        // Auditoria automatica - actualiza UpdatedAt
        foreach (EntityEntry<IAuditable> entry in
            context.ChangeTracker.Entries<IAuditable>())
        {
            if (entry.State == EntityState.Modified)
                entry.Entity.UpdatedAt = now;
        }

        // Soft Delete - convierte Delete en Modified
        foreach (EntityEntry<ISoftDeletable> entry in
            context.ChangeTracker.Entries<ISoftDeletable>())
        {
            if (entry.State != EntityState.Deleted)
                continue;

            entry.State = EntityState.Modified;
            entry.Entity.IsDeleted = true;
            entry.Entity.DeletedAt = now;
        }

        return base.SavingChangesAsync(eventData, result, cancellationToken);
    }
}
```

10.3 ChangeOrderDbContext

El DbContext aplica todas las configuraciones del assembly automaticamente via ApplyConfigurationsFromAssembly:

```
namespace ChangeOrder.Data.Context;

public sealed class ChangeOrderDbContext(DbContextOptions<ChangeOrderDbContext>
options)
    : DbContext(options)
{
    public DbSet<ChangeOrderEntity> ChangeOrders => Set<ChangeOrderEntity>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ChangeOrderDbContext).Assembly);
    }
}
```

10.4 Registro en ServiceCollectionExtensions

Registrar el interceptor como Singleton y el DbContext en la capa Data:

```
// En ServiceCollectionExtensions.cs (Data layer)
public static IServiceCollection AddDataServices(
    this IServiceCollection services,
    string connectionString)
{
    services.AddSingleton<AuditInterceptor>();

    services.AddDbContext<ChangeOrderDbContext>((sp, options) =>
        options.UseSqlServer(connectionString)
            .AddInterceptors(sp.GetRequiredService<AuditInterceptor>()));

    services.AddScoped<IChangeOrderRepository, ChangeOrderRepository>();
    services.AddScoped<IUnitOfWork, ChangeOrderDbContext>();
    return services;
}
```

10.3 Connection String

```
// appsettings.Development.json
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=ChangeOrderDb;
    Trusted_Connection=true;TrustServerCertificate=true;"
  }
}
```

11. Logging — Serilog

11.1 Niveles de Log

Nivel	Cuando Usar
Information	Operaciones exitosas (orden creada, actualizada)
Warning	Situaciones inesperadas pero manejables (orden no encontrada)
Error	Excepciones capturadas
NUNCA loggear	Datos sensibles (emails completos, datos personales sin enmascarar)

Siempre usar logging estructurado:

```
logger.LogInformation("Orden {OrderNumber} creada para {Department}",  
    order.Number.Value, order.Requester.Department);
```

11.2 Configuración appsettings.json

```
{  
  "Serilog": {  
    "Using": ["Serilog.Sinks.Console", "Serilog.Sinks.File"],  
    "MinimumLevel": {  
      "Default": "Information",  
      "Override": {  
        "Microsoft": "Warning",  
        "System": "Warning"  
      }  
    },  
    "WriteTo": [  
      { "Name": "Console" },  
      { "Name": "File", "Args": {  
        "path": "logs/log-.txt",  
        "rollingInterval": "Day"  
      }  
    }  
  ]  
}
```

12. OpenAPI / Swagger

- OpenAPI 3.1 (default en .NET 10).
- Habilitar XML doc comments en .csproj:
`<GenerateDocumentationFile>true</GenerateDocumentationFile>`.
- Usar `.WithOpenApi()` y `.WithTags()` en cada grupo de endpoints.
- Documentar cada endpoint con XML comments (`/// <summary>`).

13. Docker

13.1 Dockerfile Multi-Stage

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base
WORKDIR /app
EXPOSE 8080

FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore
RUN dotnet publish src/ChangeOrder.Host/ChangeOrder.Host.csproj \
    -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "ChangeOrder.Host.dll"]
```

13.2 .dockerignore

```
**/bin/
**/obj/
**/publish/
**/.git
**/.vs
```

Despliegue

La API se desplegará en un servidor Docker interno de la compañía (on-premises, no cloud). El contenedor corre en la red interna corporativa. No está expuesto a internet. CORS debe permitir orígenes de las máquinas internas donde correrá el cliente WPF.

14. CI/CD — GitHub Actions

- Trigger: push a 'main', PRs a 'main'.
- Pasos: restore -> build -> test -> (opcional) publish.
- Validar que ningun archivo .cs exceda 500 lineas.
- Archivo: .github/workflows/ci.yml

15. Git — Convenciones

15.1 Ramas

Rama	Proposito
main	Produccion estable — protegida, requiere PR aprobado
feature/descripcion	Nueva funcionalidad
fix/descripcion	Correccion de bug
release/vX.Y.Z	Preparacion de release

15.2 Commits — Conventional Commits

```
feat(orders): agrega endpoint de creacion de orden
fix(data): corrige mapping de ApprovalChain
chore(host): actualiza configuracion de Serilog
docs(readme): agrega instrucciones de Docker
refactor(business): extrae OrderNumberGenerator a servicio
test(business): agrega tests para CreateOrderHandler
```

- SIEMPRE merge commit (--no-ff) para preservar historial.
- NO hacer git add o git commit sin revision previa.
- PRs obligatorios para merge a main.
- Actualizar CHANGELOG.md en CADA PR antes de merge.

16. Testing

16.1 Frameworks

Framework	Proposito
xUnit	Tests unitarios
FluentAssertions	Assertions legibles y expresivas
NSubstitute	Mocking de dependencias
Microsoft.AspNetCore.Mvc.Testing	Tests de integracion para endpoints

16.2 Convenciones

Nombre del metodo: MetodoBajoPrueba_Escenario_ResultadoEsperado

```
public async Task HandleAsync_OrderExists_ReturnsSuccess()  
public async Task HandleAsync_OrderNotFound_ReturnsFailure()
```

- Un proyecto de test por capa.
- Minimo: tests para TODOS los Handlers (Commands y Queries).

17. Paquetes NuGet

ChangeOrder.Host

- Serilog.AspNetCore
- Serilog.Sinks.Console
- Serilog.Sinks.File
- Asp.Versioning.Http
- ASP.NET Core Health Checks (SqlServer)
- Microsoft.EntityFrameworkCore.Design (herramientas)

ChangeOrder.Data

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools

ChangeOrder.Presentation

- Microsoft.AspNetCore.OpenApi

Tests

- xUnit
- FluentAssertions
- NSubstitute
- Microsoft.AspNetCore.Mvc.Testing

Mappers — Regla Importante

⚠ CRITICO: Usar mapeo MANUAL con metodos estaticos de extension. NO usar AutoMapper ni Mapster. Mantener control explicito y evitar 'magia' que confunda al equipo.

```
public static class OrderMapper
{
    public static OrderResponse ToResponse(ChangeOrderEntity entity) => new(
        Id: entity.Id,
        OrderNumber: entity.Number.Value,
        ProgramName: entity.ProgramName,
        RequestDate: entity.RequestDate,
        RequesterName: entity.Requester.Name,
        Status: entity.Status.ToString());

    public static IReadOnlyList<OrderResponse> ToResponseList(
        IEnumerable<ChangeOrderEntity> entities) =>
        entities.Select(ToResponse).ToList().AsReadOnly();
}

// Regla: Un mapper por entidad.
// Regla: Nunca poner logica de negocio en el mapper.
```

18. Archivos de Configuración

18.1 Directory.Build.props

Archivo en la raíz de la solución para configuración compartida entre todos los proyectos:

```
<Project>
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>14</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <GenerateDocumentationFile>true</GenerateDocumentationFile>
  </PropertyGroup>
</Project>
```

18.2 .editorconfig

Archivo en la raíz para forzar estilo de código automáticamente. Configuraciones clave:

- indent_style = space, indent_size = 4
- csharp_style_namespace_declarations = file_scoped:error
- csharp_style_var_for_built_in_types = false:warning
- csharp_style_var_when_type_is_apparent = true:suggestion
- dotnet_style_require_accessibility_modifiers = always:warning

19. Idempotencia en POST

Para prevenir creacion de ordenes duplicadas por reintentos del cliente:

- El cliente envia un header Idempotency-Key: <GUID> en cada POST.
- El servidor verifica si ya existe una orden con esa key antes de crear.
- Implementar como middleware o filtro en el endpoint de creacion.

```
public sealed record CreateOrderRequest(  
    Guid IdempotencyKey,    // enviado por el cliente  
    string ProgramName,  
    string ProductionVersion  
    // ... demas campos  
);
```

20. Repositorio GitHub

- Rama 'main' protegida: requiere PR aprobado para merge.
- Merge siempre con --no-ff (merge commit) para preservar historial.
- CI/CD (GitHub Actions) corre en cada push y PR a 'main'.
- Releases se crean con tags versionados (vX.Y.Z) y gh release create.
- Cada capa del proyecto debe tener su propio README.md describiendo su proposito.

20.1 CHANGELOG.md

Obligatorio mantener CHANGELOG.md en la raiz siguiendo el estandar Keep a Changelog. Categorías permitidas:

Categoría	Cuando Usar
Added	Funcionalidad nueva
Changed	Cambios en funcionalidad existente
Deprecated	Funcionalidad que sera removida en el futuro
Removed	Funcionalidad eliminada
Fixed	Correccion de bugs
Security	Correcciones de vulnerabilidades

- Actualizar CHANGELOG.md en CADA PR antes de merge.
- La seccion [Unreleased] acumula cambios hasta el proximo release.
- Usar Semantic Versioning: MAJOR.MINOR.PATCH.

21. Checklist Pre-PR

Instrucciones

TODOS los puntos de este checklist deben estar completados antes de abrir un Pull Request. El revisor verificara cada punto.

1. El codigo compila sin errores ni warnings.
2. Ningun archivo .cs excede 500 lineas.
3. Constructores y metodos tienen maximo 3 parametros.
4. Se usan tipos explicitos donde el tipo no es evidente (var solo cuando es obvio).
5. No se usa 'else' — se aplica Early Return / Guard Clauses.
6. Todos los metodos async reciben CancellationToken.
7. Los tests pasan (dotnet test).
8. Se usaron TypedResults en los endpoints.
9. Los errores de negocio usan Result<T, E>, no excepciones.
10. Logging apropiado (sin datos sensibles).
11. Commit message sigue Conventional Commits.
12. Las dependencias entre capas respetan la arquitectura Onion.
13. Endpoints de listas usan paginacion (PagedResponse<T>).
14. Mappers son manuales, sin logica de negocio.
15. CHANGELOG.md actualizado con los cambios de este PR.